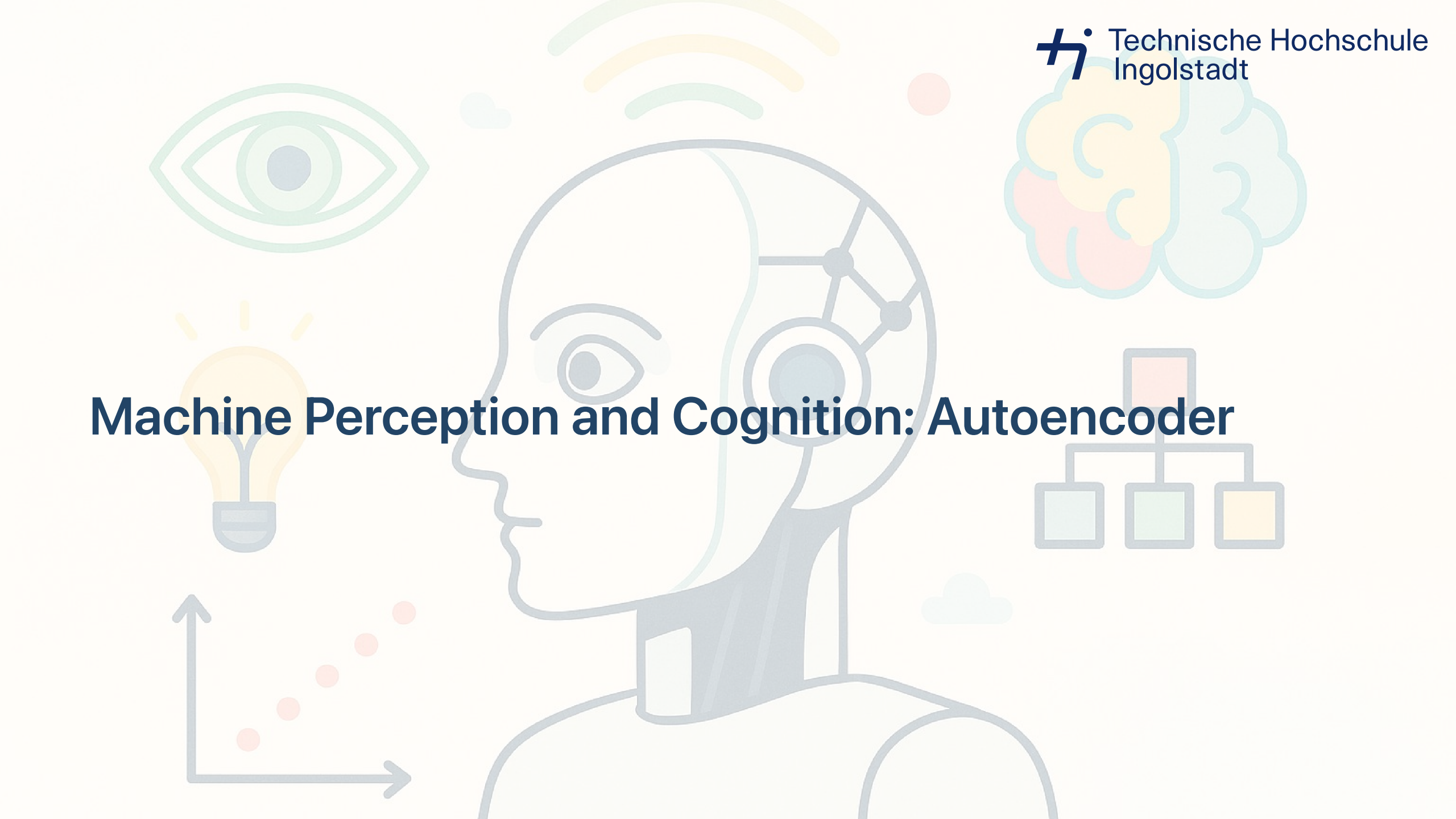
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Autoencoder

Learning Goals

After this lecture you will be able to:

- Describe the **encoder–decoder architecture** of an autoencoder and explain the role of the bottleneck
- Formulate the **reconstruction loss** for an autoencoder
- Explain how the autoencoder architecture extends the idea of **sparse coding** from the previous lecture to an end-to-end trained neural network
- Design and train a **denoising autoencoder** and explain why noise injection improves generalization
- Build a **convolutional autoencoder** for image reconstruction and compression
- Use the reconstruction error as an **anomaly score** for unsupervised anomaly detection

Why Learn Representations?

Raw data is high-dimensional but its true structure is low-dimensional:

- A 224×224 RGB image has $150\,528$ pixel values — yet images of the same scene differ only along a handful of factors (pose, lighting, identity, ...)
- **Redundancy**: neighboring pixels are highly correlated; most variation lies on a low-dimensional manifold
- Useful representations should be **compact**, **informative**, and **task-agnostic**

Why Learn Representations?

Goal: learn a mapping $\mathbf{z} = f_\theta(\mathbf{x})$ to a compact latent code $\mathbf{z}$, such that $\mathbf{x}$ can be faithfully reconstructed from $\mathbf{z}$ alone.

Connection to the previous lecture: sparse coding represented image patches as $\mathbf{x} \approx \mathbf{D}\mathbf{c}$ using a learned dictionary $\mathbf{D}$ and a sparse coefficient vector $\mathbf{c}$. The autoencoder generalizes this idea: the encoder f_ϕ replaces the expensive iterative pursuit algorithm, and the decoder g_ψ replaces the fixed linear synthesis step — both implemented as learnable neural networks.

The Autoencoder Idea

An **autoencoder** is an unsupervised neural network that learns to:

1. **Compress** the input $\mathbf{x}$ into a compact latent code $\mathbf{z}$ — the *encoder*
2. **Reconstruct** the input from the latent code — the *decoder*

The network is trained to minimize the difference between the input and its reconstruction. No labels are needed — the input itself is the target.

The Autoencoder Idea

Autoencoder, specifically a variational autoencoder:

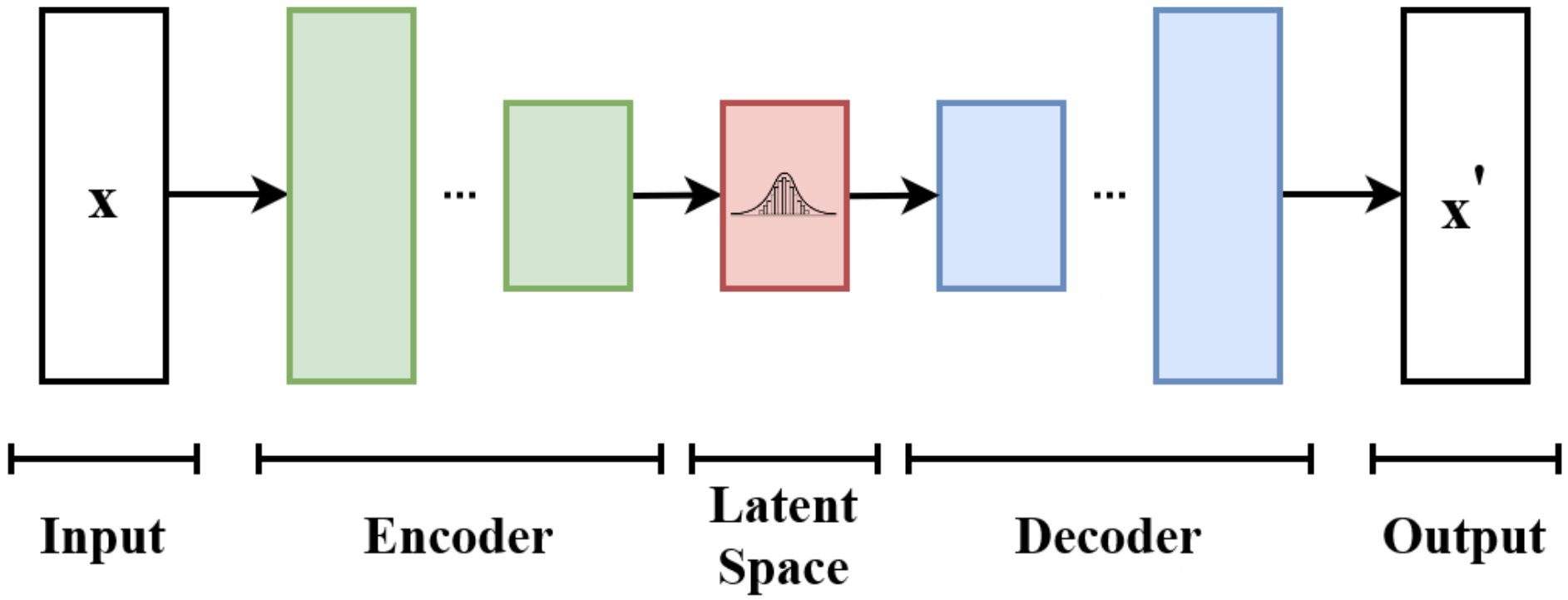


Image source: EugenioTL, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:VAE_Basic.png

Autoencoder: Formal Definition

Let $\mathbf{x} \in \mathbb{R}^D$ be an input (e.g. a flattened image).

Encoder $f_\phi : \mathbb{R}^D \to \mathbb{R}^d$:

$$\mathbf{z} = f_\phi(\mathbf{x}), \quad d \ll D$$

Decoder $g_\psi : \mathbb{R}^d \to \mathbb{R}^D$:

$$\hat{\mathbf{x}} = g_\psi(\mathbf{z}) = g_\psi(f_\phi(\mathbf{x}))$$

Reconstruction loss (MSE):

$$\mathcal{L}(\phi, \psi) = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}^{(i)} - g_\psi(f_\phi(\mathbf{x}^{(i)}))\|_2^2$$

The parameters ϕ (encoder) and ψ (decoder) are learned jointly by minimizing $\mathcal{L}$.

The Bottleneck

The **bottleneck** is the key ingredient: by restricting the latent dimension $d \ll D$, the autoencoder is forced to learn a compressed representation.

Quantity	Typical value
Input dimension D	e.g. 784 (MNIST 28×28)
Bottleneck dimension d	e.g. 32 or 64
Compression ratio D/d	e.g. $784/32 \approx 24\times$

Without the bottleneck the autoencoder could trivially learn the identity mapping — the latent code would simply copy the input, learning nothing useful.

Undercomplete vs. Overcomplete Autoencoders

Type	Bottleneck size	Key property
Undercomplete	$d < D$	Forced compression; latent manifold structure
Complete	$d = D$	No compression; needs regularization
Overcomplete	$d > D$	More expressive; must regularize heavily

This lecture focuses on **undercomplete** autoencoders (compression by bottleneck) and their regularized extensions.

The Reconstruction Loss

For real-valued inputs (images, sensor data), we use the **Mean Squared Error (MSE)**:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|_2^2$$

For binary or Bernoulli-distributed inputs (e.g. binarized images), the **Binary Cross-Entropy (BCE)** is more appropriate:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{D} \left[x_j^{(i)} \log \hat{x}_j^{(i)} + (1 - x_j^{(i)}) \log(1 - \hat{x}_j^{(i)}) \right]$$

Both losses measure how well the decoder reconstructs the original input.

Training the Autoencoder

Training follows **standard backpropagation** and gradient descent:

1. Forward pass: $\hat{\mathbf{x}} = g_\psi(f_\phi(\mathbf{x}))$
2. Compute loss: $\mathcal{L}(\phi, \psi) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$
3. Backward pass: compute $\frac{\partial \mathcal{L}}{\partial \phi}$ and $\frac{\partial \mathcal{L}}{\partial \psi}$
4. Update: $\phi \leftarrow \phi - \eta \frac{\partial \mathcal{L}}{\partial \phi}, \psi \leftarrow \psi - \eta \frac{\partial \mathcal{L}}{\partial \psi}$

Key observations:

- **No labels needed** — the target is the input itself: $\mathbf{y} = \mathbf{x}$
- The entire network is differentiable — standard Adam or SGD applies
- The encoder and decoder weights are trained **jointly**

Autoencoder as a Function Composition

The autoencoder defines a **reconstruction function**:

$$\hat{\mathbf{x}} = (g_\psi \circ f_\phi)(\mathbf{x})$$

The composition $g_\psi \circ f_\phi : \mathbb{R}^D \to \mathbb{R}^D$ is trained to approximate the **identity mapping** under a bottleneck constraint.

If the bottleneck has dimension d, the autoencoder must learn the d most informative directions of variation in the data — analogous to how sparse coding learns a compact dictionary, but realized as an end-to-end trained neural network whose parameters are optimized jointly by backpropagation.

Autoencoder as a Neural Network

An autoencoder is a standard feedforward network with two symmetrical parts:

Encoder — a deep MLP (or CNN for images) that progressively compresses the input:

$$\mathbf{h}_k = \sigma(\mathbf{W}_k \mathbf{h}_{k-1} + \mathbf{b}_k), \qquad \mathbf{z} = \mathbf{W}_K \mathbf{h}_{K-1} + \mathbf{b}_K$$

Decoder — a symmetric network that progressively expands the latent code:

$$\mathbf{h}'_k = \sigma(\mathbf{W}'_k \mathbf{h}'_{k-1} + \mathbf{b}'_k), \qquad \hat{\mathbf{x}} = \sigma'(\mathbf{W}'_K \mathbf{h}'_{K-1} + \mathbf{b}'_K)$$

All weights $\{\mathbf{W}_k, \mathbf{b}_k, \mathbf{W}'_k, \mathbf{b}'_k\}$ are **learned jointly** by backpropagation through the full encoder–decoder stack — no iterative pursuit algorithm is needed.

Image reconstruction from Autoencoders

Images reconstructed from PCA and autoencoder codes:

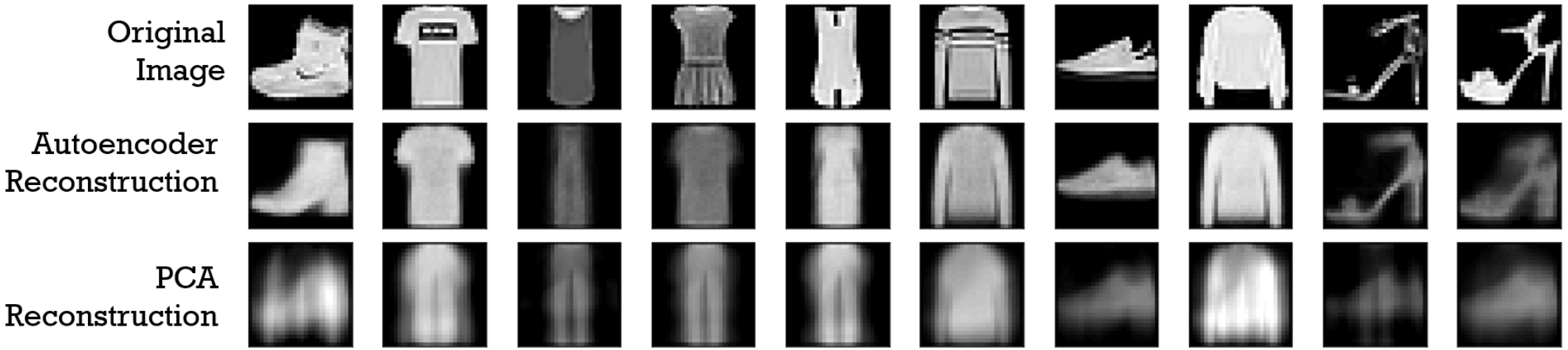


Image source: Michela Massi, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Reconstruction_autoencoders_vs_PCA.png

Why Nonlinearity and Depth Matter

Without nonlinear activations σ, the encoder and decoder collapse to a single linear projection — the latent code can only represent a **flat linear subspace**. Adding nonlinearity unlocks strictly richer representations:

Property	Linear AE	Shallow Nonlinear AE	Deep Nonlinear AE
Latent manifold	Flat hyperplane	Mildly curved	Highly curved
Captures	Variance directions	Simple nonlinear structure	Hierarchical features
Expressiveness	Low	Medium	High

The Latent Space

The latent code $\mathbf{z} \in \mathbb{R}^d$ is the **internal representation** learned by the encoder.

Properties of a well-trained latent space:

- **Compact**: similar inputs map to nearby codes
- **Smooth**: small changes in $\mathbf{z}$ produce small changes in $\hat{\mathbf{x}}$
- **Structured**: semantic categories cluster naturally

The latent space is the foundation of all downstream applications of autoencoders.

Latent Space Visualization

Training a 2D bottleneck autoencoder on Fashion-MNIST and plotting $\mathbf{z} = f_\phi(\mathbf{x})$:

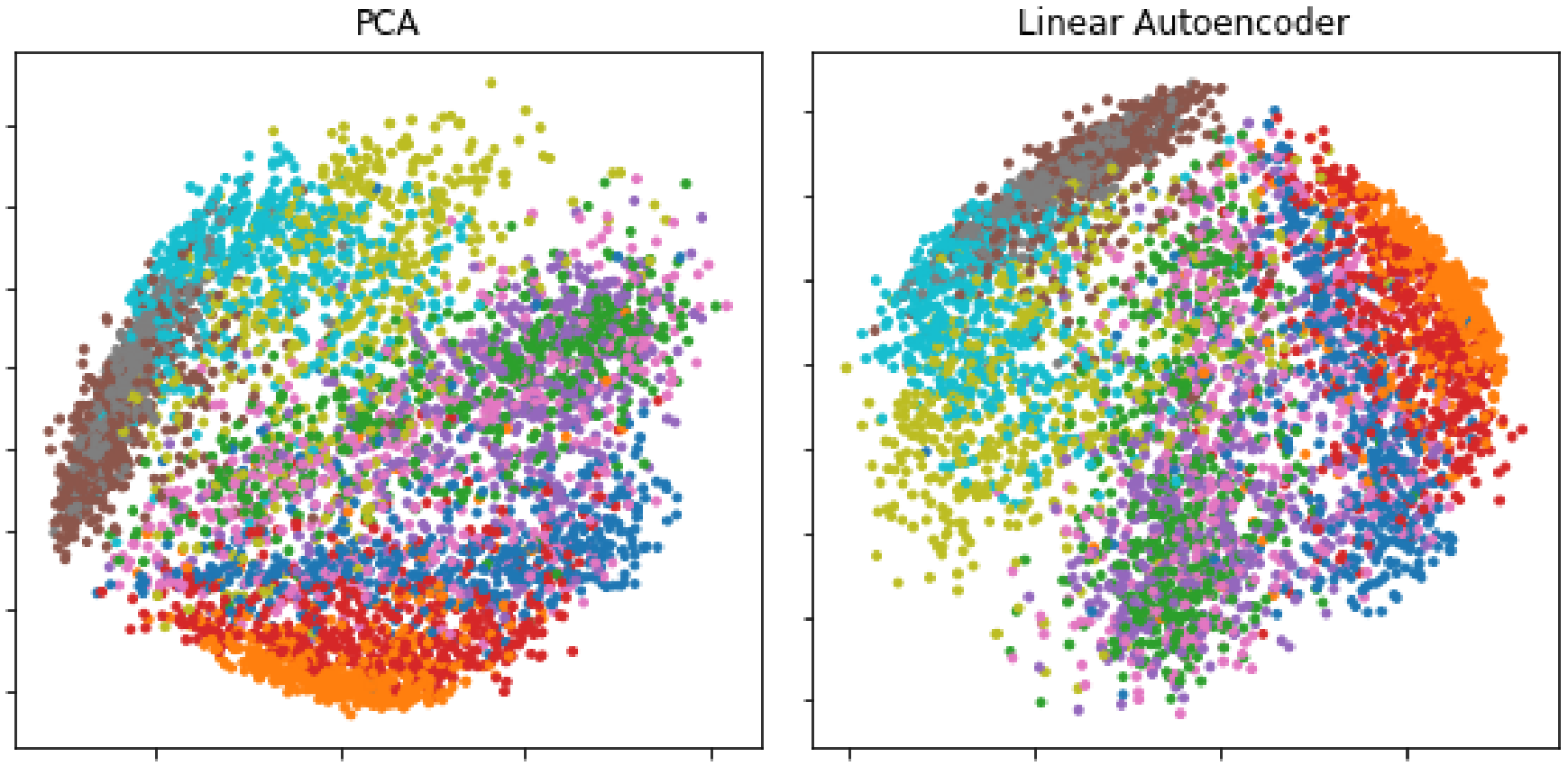


Image source: Michela Massi, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:PCA_vs_Linear_Autoencoder.png

Autoencoder as Feature Extractor

After training, **discard the decoder** and use the encoder as a fixed feature extractor:

$$\mathbf{z} = f_\phi(\mathbf{x}) \in \mathbb{R}^d$$

The latent code $\mathbf{z}$ is a **dense, low-dimensional** descriptor of $\mathbf{x}$.

Applications:

- **Clustering**: apply k-means in $\mathbb{R}^d$ instead of $\mathbb{R}^D$
- **Nearest-neighbor search**: similarity in latent space approximates semantic similarity
- **Supervised fine-tuning**: attach a small classifier head on top of the frozen encoder

Encoder as Pre-Training

The autoencoder can be used for **semi-supervised learning**:

1. **Pre-train** the encoder f_ϕ on a large **unlabeled** dataset using the reconstruction loss
2. **Fine-tune** the encoder (or a linear head on top of it) on a small **labeled** dataset

This is advantageous when labels are scarce but unlabeled data is abundant — a common scenario in medical imaging, satellite imagery, or industrial inspection.

The encoder learns general-purpose features from the structure of the data, before any labels are seen.

Clustering in Latent Space

Workflow:

1. Train autoencoder on raw data (no labels)
2. Encode all samples: $\mathbf{z}^{(i)} = f_\phi(\mathbf{x}^{(i)})$
3. Run k-means in $\mathbb{R}^d$: assign each $\mathbf{z}^{(i)}$ to cluster k^*

Why this works:

- The reconstruction loss forces the encoder to retain class-discriminative information
- The bottleneck collapses within-class variability, separating classes in $\mathbb{R}^d$
- Clustering in $d \ll D$ dimensions avoids the **curse of dimensionality**

Transfer Learning with Autoencoders

An autoencoder trained on a source domain (e.g. natural images) can be fine-tuned on a target domain (e.g. medical scans):

Step	Action
1. Pre-train	Train autoencoder on large unlabeled source dataset
2. Freeze encoder	Fix ϕ; discard decoder
3. Attach head	Add small MLP or linear classifier on $\mathbf{z}$
4. Fine-tune	Train head on small labeled target dataset

This **transfer** leverages the general low-level features (edges, textures) already learned, requiring far fewer labeled examples in the target domain.

From Sparse Coding to Autoencoders

Recall from the previous lecture: sparse coding represents patches as $\mathbf{x} \approx \mathbf{D}\mathbf{c}$, minimizing reconstruction error subject to a sparsity constraint on $\mathbf{c}$.

An autoencoder generalizes this idea to an **end-to-end trainable neural network**:

Property	Sparse Coding	Autoencoder
Encoder	Iterative pursuit (OMP)	$\mathbf{z} = f_\phi(\mathbf{x})$ — single forward pass
Decoder	$\hat{\mathbf{x}} = \mathbf{D}\mathbf{c}$ — linear	$\hat{\mathbf{x}} = g_\psi(\mathbf{z})$ — nonlinear
Dictionary / weights	Alternating updates	Learned end-to-end by backprop
Nonlinearity	None (linear synthesis)	Multiple nonlinear layers
Inference cost	Expensive iterative solve	Fast single forward pass

The Manifold Hypothesis

Manifold hypothesis: real-world high-dimensional data $\mathbf{x} \in \mathbb{R}^D$ concentrates near a low-dimensional **smooth manifold** $\mathcal{M}$ embedded in $\mathbb{R}^D$.

The autoencoder learns an **approximate coordinate system** for $\mathcal{M}$:

- Encoder f_ϕ: maps $\mathbf{x} \in \mathcal{M} \subset \mathbb{R}^D$ to coordinates $\mathbf{z} \in \mathbb{R}^d$
- Decoder g_ψ: maps coordinates back to $\mathcal{M}$

The reconstruction error $\left\|\mathbf{x} - \hat{\mathbf{x}}\right\|_2$ measures how far $\mathbf{x}$ is from the learned manifold — the basis for anomaly detection.

Denoising Autoencoder

A **denoising autoencoder (DAE)** learns to reconstruct a **clean** input from a **corrupted** version:

$$\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$$

The training objective becomes:

$$\mathcal{L}_{\text{DAE}}(\phi, \psi) = \frac{1}{N} \sum_{i=1}^{N} \| \mathbf{x}^{(i)} - g_\psi(f_\phi(\tilde{\mathbf{x}}^{(i)})) \|_2^2$$

The input to the encoder is the **noisy** sample $\tilde{\mathbf{x}}$, but the target is the **clean** sample $\mathbf{x}$.

Denoising Autoencoder: Noise Types

Different types of corruption can be applied:

Noise type	Definition	Typical use
Additive Gaussian	$\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}, \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$	Images, sensors
Masking / Dropout	Set random fraction p of entries to zero	Text, images
Salt-and-pepper	Set random entries to min or max	Binary images
Blurring	Convolve with Gaussian kernel	Images

The network must learn to **undo** the corruption — it has to understand the underlying data structure.

Denoising Autoencoder

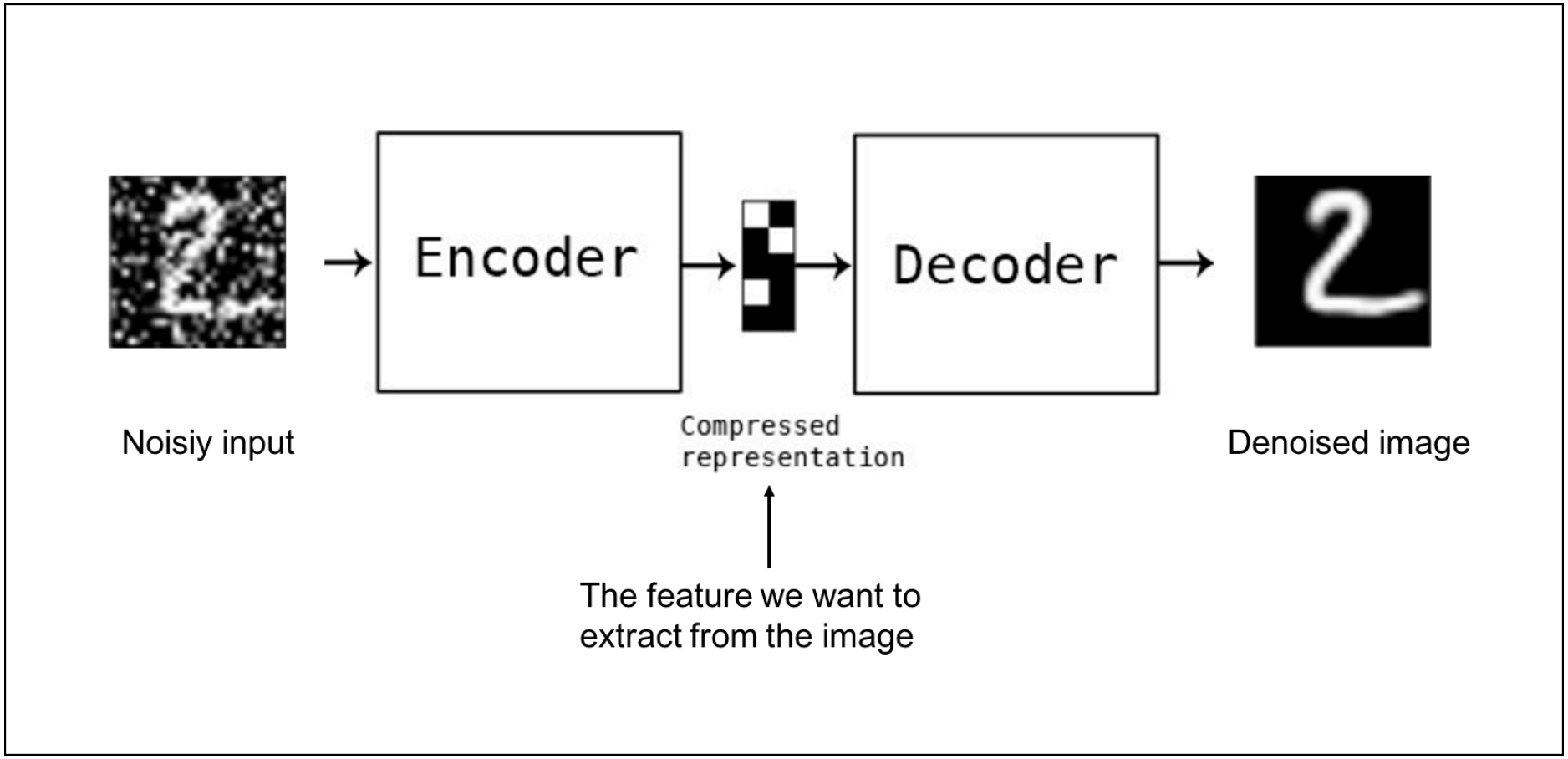


Image source: neerajkrbansal1996, CC0, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Denoising-autoencoder.png

Why Denoising Improves Generalization

Training with corrupted inputs forces the encoder to learn **robust features**:

- The network cannot memorize individual pixel values — it must capture **statistical structure**
- The corruption acts as a form of **data augmentation** and implicit regularization

DAE: Geometric Intuition

Think of the data manifold $\mathcal{M}$ embedded in $\mathbb{R}^D$:

- Clean samples $\mathbf{x}$ lie **on** $\mathcal{M}$
- Corrupted samples $\tilde{\mathbf{x}}$ lie **off** $\mathcal{M}$ (perturbed away)
- The DAE learns a **projection** from $\tilde{\mathbf{x}}$ back onto $\mathcal{M}$

The reconstruction $\hat{\mathbf{x}} = g_\psi(f_\phi(\tilde{\mathbf{x}}))$ is the network's estimate of the nearest point on $\mathcal{M}$ to $\tilde{\mathbf{x}}$.

Stronger noise σ forces the network to learn larger-scale manifold structure; weaker noise captures finer local details.

DAE: Practical Applications

Application	How DAE is used
Image denoising	Train on clean images; corrupt with Gaussian noise; infer on noisy images
Image inpainting	Mask (zero out) a region; train to reconstruct missing pixels
Speech enhancement	Add babble or white noise; train to recover clean speech signal
Self-supervised pre-training	Learn encoder on unlabeled data via denoising; fine-tune with labels

In all cases the key principle is the same: **corrupt → encode → decode → compare to clean**.

DAE vs. Standard AE

Property	Standard AE	Denoising AE
Input	Clean $\mathbf{x}$	Corrupted $\tilde{\mathbf{x}}$
Target	$\mathbf{x}$	$\mathbf{x}$
Regularization	Bottleneck only	Bottleneck + noise
Robustness	Lower	Higher
Learned features	Compact	Robust and compact
Risk of trivial solution	Higher (overfit)	Lower

The denoising objective is a powerful alternative to explicit regularization — it achieves similar or better generalization without adding penalty terms to the loss.

Summary: Denoising Autoencoder

- The DAE receives a **corrupted** input $\tilde{\mathbf{x}}$ and learns to reconstruct the **clean** $\mathbf{x}$
- Noise injection is a powerful form of **implicit regularization**: the network must learn robust features
- The DAE gradient approximates the **score function** of the data distribution — a connection to diffusion models
- Applications: image denoising, speech enhancement, image inpainting, self-supervised learning

Regularized Autoencoders

Sparse autoencoder: add an ℓ_1 penalty on the latent activations:

$$\mathcal{L}_{\text{sparse}}(\phi, \psi) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda \|\mathbf{z}\|_1$$

The ℓ_1 term encourages most latent units to be inactive for any given input — only a small subset $\mathbf{z}_k \neq 0$ encodes the input. This is the **neural network analogue of sparse coding**.

Deep Autoencoders

A **deep autoencoder** stacks multiple layers in both encoder and decoder:

Encoder: $\mathbf{x} \to \mathbf{h}_1 \to \mathbf{h}_2 \to \cdots \to \mathbf{z}$

Decoder: $\mathbf{z} \to \mathbf{h}'_{K-1} \to \cdots \to \mathbf{h}'_1 \to \hat{\mathbf{x}}$

Each layer applies an affine transformation followed by a non-linear activation σ:

$$\mathbf{h}_{k+1} = \sigma(\mathbf{W}_k \mathbf{h}_k + \mathbf{b}_k)$$

Benefits of depth:

- Hierarchical feature extraction (edges → textures → parts → objects)
- More expressive encoder for complex data distributions

Convolutional Autoencoder

For image data, fully-connected layers are inefficient. A **convolutional autoencoder (CAE)** uses:

- **Encoder**: convolutional layers + pooling (e.g. max-pool or stride-2 conv) to **downsample** the spatial resolution
- **Decoder**: transposed convolutional layers or upsampling + conv to **upsample** back to the original resolution

Convolutions exploit the spatial structure of images:

- Shared weights across spatial positions (**translation equivariance**)
- Local receptive fields (**locality bias**)
- Far fewer parameters than fully connected layers

Convolutional Autoencoder Architecture

$$\underbrace{(H \times W \times C)}_{\mathbf{x}} \xrightarrow{\text{Conv + Pool}} \underbrace{(H/s \times W/s \times C')}_{\mathbf{h}} \xrightarrow{\text{Conv + Pool}} \underbrace{(d)}_{\mathbf{z}} \xrightarrow{\text{TConv}} \underbrace{(H/s \times W/s \times C')}_{\hat{\mathbf{h}}} \xrightarrow{\text{TConv}} \underbrace{(H \times W \times C)}_{\hat{\mathbf{x}}}$$

- **Stride-2 convolution** halves spatial size while increasing channel depth
- **Transposed convolution** (also called deconvolution) doubles spatial size
- The bottleneck $\mathbf{z}$ is typically a flattened feature map or a global average pooled vector

Transposed Convolution (Deconvolution)

A **transposed convolution** is a learnable operation that can be used to **upsample** feature maps.

For a regular convolution with kernel $\mathbf{W}$, stride s, input size n:

$$\text{output size} = \lfloor (n - k)/s \rfloor + 1$$

For the corresponding transposed convolution (same $\mathbf{W}$, stride s):

$$\text{output size} = (n - 1) \cdot s + k$$

The weights $\mathbf{W}$ of the transposed convolution are **learned** during training, enabling the network to learn task-specific upsampling.

Upsampling Methods in Decoders

Method	Learnable	Artifacts	Typical use
Nearest-neighbor	❌	Blocking	Cheap baseline
Bilinear interpolation	❌	Blurring	Segmentation decoders
Transposed convolution	✅	Checkerboard	GANs, autoencoders

The **transposed convolution** is the most widely used learnable upsampling strategy in autoencoders and generative models.

Convolutional AE: Encoder in Detail

Encoder (example for 28×28 grayscale input):

Layer	Output shape	Operation
Input	$28 \times 28 \times 1$	—
Conv2D (32 filters, 3×3, stride 2)	$14 \times 14 \times 32$	Downsample
BatchNorm + ReLU	$14 \times 14 \times 32$	Normalize + activate
Conv2D (64 filters, 3×3, stride 2)	$7 \times 7 \times 64$	Downsample
BatchNorm + ReLU	$7 \times 7 \times 64$	Normalize + activate
Flatten + Linear(d)	d	Bottleneck

Total parameters much smaller than a fully-connected equivalent.

Convolutional AE: Decoder in Detail

Decoder (symmetric to the encoder):

Layer	Output shape	Operation
Linear($7 \cdot 7 \cdot 64$) + Reshape	$7 \times 7 \times 64$	Expand bottleneck
TConv2D (32 filters, 3×3, stride 2)	$14 \times 14 \times 32$	Upsample
BatchNorm + ReLU	$14 \times 14 \times 32$	Normalize + activate
TConv2D (1 filter, 3×3, stride 2)	$28 \times 28 \times 1$	Upsample
Sigmoid	$28 \times 28 \times 1$	Normalize to $[0, 1]$

The decoder mirrors the encoder structure — deeper and wider networks produce higher-quality reconstructions.

Anomaly Detection

An autoencoder trained only on **normal** data learns to reconstruct normal samples well. Anomalies are poorly reconstructed because they lie **off the learned manifold**.

Anomaly score:

$$s(\mathbf{x}) = \|\mathbf{x} - g_\psi(f_\phi(\mathbf{x}))\|_2^2$$

Decision rule with threshold τ:

$$\text{anomaly if } s(\mathbf{x}) > \tau$$

The threshold τ is set on a validation set of normal samples (e.g. at the 95th or 99th percentile of s).

Anomaly Detection: Manifold Interpretation

Sample type	Reconstruction quality	Score $s(\mathbf{x})$
Normal (in-distribution)	High	Low
Anomaly (out-of-distribution)	Low	High

The autoencoder acts as a **learned density estimator**: it models the manifold of normal data, and the reconstruction error is a proxy for $-\log p(\mathbf{x})$.

Key assumption: the anomaly is sufficiently different from normal samples that it cannot be well-decoded from the bottleneck.

Anomaly Detection: Industrial Inspection

A prominent real-world application is **visual surface inspection**:

1. Collect thousands of images of **defect-free** products (easy, cheap)
2. Train a convolutional autoencoder on defect-free images only
3. At inference: compute $s(\mathbf{x})$ for each new product image
4. Flag images with $s(\mathbf{x}) > \tau$ as potentially defective

Advantages:

- No defect labels required for training
- Generalizes to novel defect types never seen during training
- Reconstruction difference $\mathbf{x} - \hat{\mathbf{x}}$ provides a spatial **defect map**

Limitations of Standard Autoencoders

Limitation	Explanation
No probabilistic structure	The latent space has no guaranteed distribution
Entangled representations	Different factors of variation mix in $\mathbf{z}$
No generative capability	Cannot sample new data points from a well-defined prior
Anomaly score fragility	Very powerful decoders reconstruct anomalies well too

These limitations motivate **variational autoencoders (VAEs)**, which impose a probabilistic prior on $\mathbf{z}$.

Autoencoder Variants Summary

Variant	Key idea	Typical use
Standard AE	Reconstruct input through bottleneck	Dimensionality reduction
Denoising AE	Reconstruct clean from corrupted	Denoising, pre-training
Sparse AE	ℓ_1 penalty on $\mathbf{z}$	Interpretable features
Convolutional AE	Conv + TConv architecture	Image reconstruction
Variational AE	Probabilistic latent space	Generation, disentanglement

Training Tips for Autoencoders

Practical recommendations for stable training:

- **Batch normalization** in the encoder helps stabilize training and prevents vanishing gradients
- **Adam optimizer** typically converges faster than plain SGD for autoencoders
- **Symmetric architecture**: mirroring the encoder in the decoder is a good starting point
- **Tied weights** ($\mathbf{W}_d = \mathbf{W}_e^\top$): halves the parameter count, often improves generalization
- **Warm-up**: start with high learning rate and decay using cosine annealing

Autoencoder and U-Net — Connection

The **U-Net** architecture used for image segmentation is an extension of the convolutional autoencoder:

- U-Net adds **skip connections** from encoder layers to the corresponding decoder layers
- Skip connections bypass the bottleneck, preserving fine spatial detail
- The decoder can combine **coarse semantic information** from $\mathbf{z}$ with **fine spatial detail** from the skip connections

$$\hat{\mathbf{h}}_k = \mathrm{TConv}(\hat{\mathbf{h}}_{k+1}) \oplus \mathbf{h}_k$$

where $\oplus$ denotes concatenation along the channel dimension. This will be covered in detail in the U-Net lecture.

Summary: Autoencoder Architecture

- An autoencoder consists of an **encoder** f_ϕ, a **bottleneck** $\mathbf{z}$, and a **decoder** g_ψ, all implemented as neural network layers trained jointly by backpropagation
- Training minimizes the **reconstruction loss** $\|\mathbf{x} - g_\psi(f_\phi(\mathbf{x}))\|_2^2$ without any labels
- The bottleneck enforces **compression**: the network must retain only the most informative structure
- Deep nonlinear autoencoders capture **curved data manifolds** — going beyond what any linear compression method can represent

Summary: Key Use Cases

Use case	Key idea
Feature extraction	Use encoder f_ϕ as a fixed or fine-tuned feature extractor
Clustering	Apply k-means to latent codes $\mathbf{z}$
Transfer learning	Pre-train encoder on unlabeled data; fine-tune with labels
Image denoising	Train DAE to reconstruct clean from corrupted input
Self-supervised pre-training	Use DAE loss as a proxy task for representation learning
Anomaly detection	High reconstruction error $\Rightarrow$ anomaly

Autoencoder

Recap!

- 🏗️ **Encoder–decoder architecture**: encoder f_ϕ compresses input $\mathbf{x}$ into a compact bottleneck code $\mathbf{z}$; decoder g_ψ reconstructs $\hat{\mathbf{x}}$ — trained jointly by minimizing the reconstruction loss $\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$ without any labels
- 🌐 **Manifold hypothesis**: the latent space approximates a low-dimensional manifold; reconstruction error $\|\mathbf{x} - \hat{\mathbf{x}}\|_2^2$ measures how far a sample lies from that manifold
- 🔇 **Denoising autoencoder (DAE)**: input is corrupted ($\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}$), target is the clean original — forces the encoder to learn robust features and acts as implicit regularization
- 🚨 **Anomaly detection**: train on normal data only; anomalies yield high reconstruction error $s(\mathbf{x}) = \|\mathbf{x} - g_\psi(f_\phi(\mathbf{x}))\|_2^2 > \tau$ and are flagged as out-of-distribution